I

Power-user command reference (alphabetical lookup)

> **Note**
>
> *The alphabetical reference for every power-user command introduced in Chapters 25-27. Use it as a glossary, when you see* `/rewind` *mentioned in passing six months from now and can't remember whether it rolls back one prompt or many, flip here.*
>
> *Verified against Claude Code v2.x command surface as of draft date. The Claude Code surface evolves; commands may be renamed or restructured. The errata page (*`github.com/codecoincognition/claude-code-4-non-programmers-v2/blob/main/ERRATA.md`*) is the live truth: if an entry here doesn't match what your terminal does, the errata page wins.*

How to use this appendix

Each entry has the same shape:

What it does: one paragraph naming the core function. **When to use it**: one paragraph naming the situation that calls for it. **Example**: a representative reader prompt and the response you'd see. **Related**: other commands that compose with this one. **Taught in**: the chapter that introduced the command.

For the conceptual depth (*why* `/compact` is lossy, *how* the auto-mode classifier works) go back to the chapter. This is for *which command was that one again.* The families table at the end is the reading aid for finding the right neighborhood before drilling into a specific entry.

A reminder: throughout the book, Maya mostly prompts in plain English and lets Claude pick the underlying command, that's the book's default, so you never have to memorize a name to get something done. This appendix is the flip side: when you *do* want to type a command yourself, this is where you look it up. Both ways work; reach for whichever you like.

The reference

```
/auto *
```

What it does: Switches on auto mode for the session, an AI classifier reads each prompt and picks the appropriate command, model, and effort, telling you what it picked.

When to use it: When you'd rather prompt in plain English and have Claude pick the underlying command surface, especially for review tasks where the right command varies by file type.

Example: *"*`/auto`*. Now: review the three files I authored today." "auto mode on. Routing:* `build.sh` → */simplify;* `cancel-leak.md` → */security-review; kill-*`switch.sh` → */security-review + /sandbox."*

Related: `/simplify`, `/security-review`, `/sandbox`, `/batch`, auto mode picks among these.

Taught in: *Chapter 26*.

* `/auto` is presented here as the personalized command Maya/Devon authored that wraps auto-mode behavior (*Chapter 26*). It isn't a Claude Code built-in. The built-in primitive is **auto permission mode**, set via `--permission-mode auto` or by cycling *Shift + Tab* to it. `/auto` here would invoke the underlying mode plus author-chosen review wrapper.

```
/batch
```

What it does: Splits a sweep into 5-30 independent units of work, dispatches each into its own isolated git worktree in parallel, and opens a pull request per unit when finished. Requires a git repo.

When to use it: Sweep operations, apply a corrective pattern (error handling, naming, dependency upgrade) across a folder of files at once, where each unit is independent enough to land as its own PR. Faster than running `/simplify` six times serially.

Example: *"*`/batch`*, apply the same error-handling pattern to all six refresher scripts in* `~/work/dashboard/refreshers/` *(one unit per file)."* Six worktrees run in parallel; six PRs opened; reader reviews each.

Related: `/simplify` (single-file deep), `/security-review` (audit broad).

Taught in: *Chapter 26*.

```
/btw
```

What it does: Mid-task correction primitive, interjects an additional constraint or instruction into a Claude reply that's already in progress, without restarting the work.

When to use it: Halfway through Claude's reply, you realize you also wanted X. Type `/btw add X` instead of waiting for the reply, then re-prompting.

Example: *"`/btw` also add error logging when the Stripe refresher fails, keep going on the rest."* Claude folds the correction into the in-progress work without losing the thread.

Related: `/rewind` (when correction is too big for `/btw` and you need to roll back), *Ctrl + G* (for editing long prompts in your editor).

Taught in: *Chapter 25*.

```
/compact
```

What it does: Summarizes the current session's history into a compact form, freeing up context-window space.

When to use it: When `/context` shows you're approaching the model's context limit. Pin irreducible decisions to a `decisions.md` file before compacting: the compactor is lossy.

Example: *"`/compact`" "context now 12% (was 67%). 47 prompts compressed."* Decisions and current state preserved; intermediate reasoning discarded.

Related: `/context` (check before you compact), `/rewind` (alternative if you only need to roll back a portion).

Taught in: *Chapter 25*.

```
/context
```

What it does: Shows the current context-window usage as a percentage of the model's window.

When to use it: When responses feel slow or you're suspicious you've been accumulating verbose history. Run alongside `/cost` for situational awareness.

Example: *"`/context`" "67% of 200k tokens used."*

Related: `/cost` (token spend), `/compact` (when context is too high).

Taught in: *Chapter 25*.

```
/cost
```

What it does: Shows the total session token spend.

When to use it: Before a long run; or when you're suspicious you've been burning tokens; or routinely at end-of-session for hygiene.

Example: *"`/cost`" "$0.43 spent over 47 prompts. Most expensive: prompt 31 (computer use, $0.08)."*

Related: `/context` (the other situational-awareness command), `/effort` (cost driver).

Taught in: *Chapter 25*.

```
Ctrl + G
```

What it does: Opens your external editor (configured via `$EDITOR`) for editing a multi-line prompt instead of typing it directly into the terminal.

When to use it: When the prompt is long, has formatting, or you want vim/emacs/VS Code's editing surface for it.

Example: *(press Ctrl + G inside a partially-typed multi-line prompt)* → Vim opens with your prompt-so-far; you finish editing; save and quit; the prompt returns to the terminal ready to send.

Related: `--resume` (session persistence).

Taught in: *Chapter 25. (Default keybinding in current Claude Code; rebindable via* `~/.claude/keybindings.json`*. Check* `/help` *or the keybindings reference if the default has changed.)*

```
/effort
```

What it does: Sets the reasoning intensity for the session. Real levels: `low` | `medium` | `high` | `xhigh` | `max` | `ultracode` | `auto`. Higher effort means deeper reasoning, more tokens, more cost.

When to use it: When the session is doing hard reasoning work and you want to bias toward depth. For one-shot deep reasoning, prefer the `ultrathink` keyword.

Example: *"*`/effort high`*" "effort set to high for the rest of the session."*

Related: `ultrathink` (one-prompt variant), `/model` (model + effort together drive cost), `/cost` (watch the spend).

Taught in: *Chapter 25.*

```
/branch
```

What it does: Switches your current session into a *copy* of the conversation up to the chosen point, a new conversation ID, with you driving. The parent conversation is left intact and resumable.

When to use it: When you want to try an alternative approach yourself and keep the original thread untouched. Return to the parent with `claude --resume <parent-id>`.

Example: *"*`/branch`*" "branched. Parent: abc123. You are now in: def456."*

Related: `/fork` (when you want a background subagent to take the alternative, not you), `--resume` (to switch back).

Taught in: *Chapter 25.*

```
/fork
```

What it does: Spawns a forked subagent in the background that inherits the full conversation and works on the directive you pass. Its result returns to your conversation when it finishes; your current session keeps going.

When to use it: When you want a parallel attempt running while you stay on the main thread. To switch yourself into a copy of the conversation instead, use `/branch`. (Note: before v2.1.161, `/fork` was an alias for `/branch`.)

Example: *"`/fork` draft three alternative outlines for the chapter, return when done." "forked subagent dispatched (def456). Result will return inline."*

Related: `/branch` (to switch yourself into the copy), `--resume` (to revisit either thread).

Taught in: *Chapter 25*.

```
--channels
```

What it does: A session-launch flag. Loads one or more channel plugins so an MCP server can *push* inbound events into the running session (the architectural inverse of every other MCP, which Claude calls into). Each value is a plugin spec: `plugin:telegram@claude-plugins-official`, `plugin:imessage@claude-plugins-official`, `plugin:discord@claude-plugins-official` are the three official Anthropic plugins; `--dangerously-load-development-channels` is required during research preview to load custom channels you authored yourself.

When to use it: When launching a session that should listen to a chat surface, Pheme's background session in *Chapter 15* is the canonical example. Add one `--channels` per channel plugin; combine with `--bg` and `--agent pheme` to run an event-driven listener that wakes on inbound messages.

Example: `claude --channels plugin:telegram@claude-plugins-official -- channels plugin:imessage@claude-plugins-official -n pheme --agent pheme --bg`

Pheme listens; Telegram + iMessage messages from allowlisted senders wake the session; he reacts and replies through the same channel.

Related: `/channels` (list active channels), `/telegram:access`, `/imessage:access`, `channelsEnabled` in `settings.json` (*Appendix F*).

Taught in: *Chapter 15*.

```
/channels
```

What it does: Lists the channel plugins active in the current session. Shows each channel's status, sender allowlist, and pairing state. Read-only, to add or remove a channel, restart the session with a different `--channels` flag.

When to use it: After launching with `--channels` to verify Pheme picked up the plugins you expected. When debugging a "Pheme didn't react" gap: a missing channel here is the first thing to check.

Example: *"`/channels`" "telegram (plugin:telegram@claude-plugins-official), paired, allowlist [Dana]. imessage (plugin:imessage@claude-plugins-official), paired, allowlist [Dana, Family]."*

Related: `--channels`, `/telegram:access`, `/imessage:access`.

Taught in: *Chapter 15*.

```
/goal
```

What it does: Sets a *condition* that Claude keeps working toward across turns until it's met. With no argument, shows the current or most recently achieved goal. The words `clear`, `stop`, `off`, `reset`, `none`, and `cancel` all remove an active goal early.

When to use it: When you know the *done-state* but not how many turns it'll take to get there, fixing a flaky test until it passes three runs in a row, refactoring until the build is green, debugging until a specific error stops appearing. The condition replaces the human nudge of *"try again."*

Example: *"*`/goal` *the orchestrator double-ping tests pass three runs in a row"* Claude iterates across turns (adjusting matchers, re-running the suite, reporting after each pass) until the condition is met or you cancel.

Related: `/loop` (session-shaped persistence), `/schedule` (clock-shaped persistence), `/effort` (raise reasoning depth for the iteration loop).

Taught in: *Chapter 25*.

```
/insights
```

What it does: Reads your session history and produces a structured report, patterns you repeat, prompts you almost-always paste, agents you almost-always invoke together, skills that would save the most time.

When to use it: Monthly. Or when you suspect you've been repeating yourself.

Example: *"*`/insights --last 30d`*" "You've prompted 'check the calendar for conflicts' as a follow-up to /triage in 12 of 18 mornings; you've prompted 'summarize my week and file under journal/' on 7 consecutive Fridays. Two patterns are skill-shaped."*

Related: *Chapter 29*'s governance hour; the manual *distill-a-pattern-into-a-skill* prompt from *Chapter 27*.

Taught in: *Chapter 27*.

`/loop` ***(bundled skill; alias*** `/proactive`***)***

What it does: Runs a prompt repeatedly while the session stays open. Omit the interval and Claude self-paces between iterations. Omit the prompt and, where available, Claude runs an autonomous maintenance check or the prompt in `.claude/loop.md`. Per docs, `/loop` is a *bundled skill*: it ships inside Claude Code's skills surface rather than as a built-in CLI command. The alias `/proactive` is interchangeable.

When to use it: When you want a recurring check that lives only as long as you're at the terminal, tailing a deploy log, polling a slow API, watching a test suite, waiting for a teammate's commit to land. Stops cleanly when the session closes. For schedules that must run whether or not you're there, use `/schedule` (cloud) or cron (local) instead.

Example: *"*`/loop` *5m check if the deploy finished"* Claude re-runs the check every five minutes; the loop persists until the session ends or you cancel.

Related: `/goal` (condition-shaped persistence), `/schedule` (clock-shaped persistence, runs without a session).

Taught in: *Chapter 25*.

```
/plugin
```

What it does: Browses, installs, and manages plugins from configured marketplaces, including the official Anthropic marketplace. Plugins can bundle skills, subagents, hooks, slash commands, and MCP servers; MCP integrations live in the *External integrations* section of `/plugin` Discover.

When to use it: When you're missing an MCP, skill, or workflow someone else has already packaged. Run Appendix C's rubric before installing.

Example: *"`/plugin`"* Interactive marketplace browser; pick *Discover → External integrations* for MCPs.

Related: `claude mcp add <name> -- <command>` (one-off MCP install without a plugin), `/skills` (list skills already loaded by installed plugins), *Appendix C* (evaluation rubric).

Taught in: *Chapter 25*.

```
/model
```

What it does: Switches the active model for the session.

When to use it: When you want a heavier model for a hard reasoning step, or a lighter model for cheap routine work.

Example: *"`/model opus`"* *"model switched to opus for the rest of the session."*

You can also pin a specific dated model ID published on Anthropic's models page, but the aliases (`opus`, `sonnet`, `haiku`) are the safer default because they follow Anthropic's most-recent release and don't go stale between minor versions.

Related: `/effort`, `ultrathink`, `/cost` (model is the largest cost driver).

Taught in: *Chapter 25*.

```
/plan
```

What it does: Enters plan mode, Claude reads, greps, and explores read-only, then drafts a single plan and waits for your approval before any execution begins. Persists until you exit with `/plan` again or *Shift* + *Tab*.

When to use it: Before risky operations: production deploys, browser-use sessions on new sites, computer-use sessions, modifying hooks.

Example: *"`/plan`"* *"plan mode on. Claude researches, drafts a plan, then offers an approval gate that exits to your chosen execution mode (auto / acceptEdits / per-edit review)."*

Related: `/auto` (auto mode includes plan-mode escalation logic), Chapter 10's audit hook.

Taught in: *Chapter 25* (surfaced as a command); Chapter 10 (taught as a permission posture).

`/review-mine` **** (personalized, not built-in)***

What it does: A personalized slash command authored in *Chapter 26* that runs the auto-mode review pipeline on whatever set of files the reader passes as `$ARGUMENTS`, filing output to `~/work/reviews/{date}/`.

When to use it: End-of-week review of files you authored that week. Wire into a Stop hook for end-of-session review.

Example: "`/review-mine ~/work/dashboard/build.sh ~/work/.claude/commands/cancel-leak.md`" Review folder populated.

Related: `/auto`, `/simplify`, `/security-review`, `/sandbox`, `/batch`.

Taught in: *Chapter 26. (Personalized command, not in the marketplace; you author it yourself in Chapter 26.)*

* This is a personalized command from this book, not a built-in. Install with the prompt in Chapter 26.

```
/rewind
```

***(also** Esc Esc*)

What it does: Rolls back the session to a chosen prior prompt. Selectively undoes; doesn't lose the rest of the session.

When to use it: When you realize three prompts ago was the wrong fork. The keyboard shortcut *Esc Esc* is the faster path: works inside an in-progress reply too. *(Default binding in current Claude Code; if your build has rebound it, check* `/help` *for the active shortcut.)*

Example: "`/rewind`" *"select prompt to roll back to: …"* (interactive selector)

Related: `/fork` (background subagent), `/branch` (switch yourself into a copy).

Taught in: *Chapter 25.*

```
--resume
```

and

```
claude rm
```

What it does: `claude --resume` opens an interactive picker of your resumable sessions and restores the one you choose. `claude rm <id>` deletes a session you no longer want. For automatic pruning, set `cleanupPeriodDays` in `settings.json`.

When to use it: `--resume` for returning to a session after closing the terminal, or for switching back to a parent thread after a `/branch` or `/fork`. `claude rm` for one-off cleanup.

Example: `claude --resume` → Interactive picker; select session; restored; prompt history visible; ready to continue.

Related: `/fork`, `/branch`, `cleanupPeriodDays` in `settings.json`.

Taught in: *Chapter 25*.

```
/sandbox
```

What it does: Runs a script or command in an isolated environment with mocked filesystem, network, and credentials so it can't touch real state.

When to use it: First run of any new shell script; first run of any Claude-authored hook; any operation that touches `~/work/` paths you can't easily restore.

Example: "`/sandbox ~/work/scripts/kill-switch.sh --simulate-call`" Script run inside isolated environment; behavior on three test cases shown.

Related: `/security-review` (audit before sandbox); `/batch` (sweep with sandbox-per-file).

Taught in: *Chapter 26. (Available on macOS, Linux, and WSL2. Native Windows is not supported.)*

```
/schedule
```

What it does: Creates *routines* that execute on Anthropic-managed cloud infrastructure on a cron schedule (alias: `/routines`). Claude walks you through setup conversationally.

When to use it: Anywhere you'd say *"refresh this every night"* or *"run this every Monday morning"* and want it to run regardless of whether your laptop is on. For schedules that must run on your local machine (read from a local file, hit `localhost`), use cron (macOS/Linux) or Task Scheduler (Windows) instead.

Example: "`/schedule refresh the dashboard every morning at 3 AM`" Claude walks through routine setup; confirms cadence; routine registered in Anthropic's cloud.

Related: the always-on hint from Chapter 9; cron / Task Scheduler for local-machine schedules.

Taught in: *Chapter 25* (surfaced); *Chapter 9* (taught conceptually); *Chapter 27* (deep dive).

```
/security-review
```

What it does: Reviews your pending git changes (the edits you've made but haven't committed yet, the same set `git diff` would show) and flags security risks: secrets, injection surfaces, auth gaps, insecure defaults, and AI-authored-code-specific bug shapes. You don't pass it a file path; it picks up whatever you've changed. (*Chapter 26* opens with a light gloss of these git words, you don't need to deeply understand them to use the command.)

When to use it: Before committing any Claude-authored work. Especially before committing slash command files (these grant permissions). Run it on the change set, not on the whole repo.

Example: "`/security-review`" (after editing `~/work/.claude/commands/cancel-leak.md` but before committing) → Categorized findings: HIGH (auto-approval threshold path), MEDIUM (no audit log), LOW (filename convention).

Related: `/simplify` (for quality, not safety), `/sandbox` (for behavior), *Chapter 10*'s audit hook (the automated cousin).

Taught in: *Chapter 26*.

Session cleanup *(not a slash command)*

There is no `/sessions cleanup` slash command. To remove a session, use `claude rm <id>` from your shell, or set `cleanupPeriodDays` in `settings.json` for auto-pruning. See `--resume and claude rm` above.

```
/simplify
```

What it does: Four-agent review pipeline focused on quality, not bugs, looks for *reuse, simplification, efficiency*, and *abstraction (altitude)* cleanups, then applies the fixes.

When to use it: When the code works but smells off, duplication, over-complication, awkward abstractions. Pair with `/code-review` if you also want correctness bugs surfaced.

Example: "`/simplify ~/work/dashboard/build.sh`" Four-agent findings; proposed diffs; applied to the working tree.

Related: `/code-review` (correctness bugs), `/security-review` (security audit), `/batch` (sweep across many files).

Taught in: *Chapter 26*.

```
/imessage:access
```

What it does: Manages the sender allowlist for the iMessage channel plugin. Subcommands include `allow <handle>` (add a phone number, Apple ID, or "Family" group to the allowlist), `revoke <handle>`, and `list` (show the current allowlist). The allowlist persists in `~/.claude/channels/imessage/access.json` and gates which inbound messages are surfaced to the channel-loaded session.

When to use it: Right after launching Pheme with `--channels plugin:imessage@...` and before relying on him to react: *every channel must be gated on a sender allowlist.* Unallowlisted messages still land in `chat.db` but are kept out of tool results, so a stranger's text can't trigger Pheme. iMessage needs no token; it reads `~/Library/Messages/chat.db` after macOS Full Disk Access is granted to Terminal.

Example: "`/imessage:access allow +14155551234`" Adds Dana's number to Pheme's iMessage allowlist; her next message wakes the session.

Related: `--channels`, `/channels`, `/telegram:access`.

Taught in: *Chapter 15. (macOS-only. iMessage channel plugin requires Full Disk Access for Terminal.)*

```
/skills
```

What it does: Lists the skills currently loaded in your session, both first-party and any shipped inside installed plugins. Skills aren't browsed from a standalone marketplace; they're distributed inside plugins.

When to use it: When you'd want to see what skills are available right now, or confirm a plugin you just installed actually contributed the skill you expected.

Example: "`/skills`" List of loaded skills with their descriptions.

Related: `/plugin` (to discover and install plugins that contain skills), *Appendix C*.

Taught in: *Chapter 25*.

```
/telegram:access
```

What it does: Manages the pairing flow and sender allowlist for the Telegram channel plugin. The canonical pair-up flow is:

1. Save your bot token (from BotFather) to `~/.claude/channels/telegram/.env` as `TELEGRAM_BOT_TOKEN=...`
2. DM your bot from your real Telegram account
3. Run `/telegram:access pair <code>` where `<code>` is the pairing code the bot replies with

After pairing, `/telegram:access allow <handle>` adds an allowed sender; `revoke` removes one; `list` shows the current state. State lives in `~/.claude/channels/telegram/access.json`.

When to use it: First-time setup of the Telegram channel (the pairing step), and any time you need to widen or tighten the allowlist. *Every channel must be gated on a sender allowlist*: an ungated Telegram channel is a documented prompt-injection vector (*Chapter 15*'s "When it goes wrong" beat).

Example: "`/telegram:access pair 482917`" Pairs your Telegram account to the bot; subsequent DMs wake Pheme's session. Follow with `allow <username>` for each additional sender (e.g., Dana).

Related: `--channels`, `/channels`, `/imessage:access`, `TELEGRAM_BOT_TOKEN` in `settings.json` env block.

Taught in: *Chapter 15*.

`ultrathink` ***(keyword, not a slash command)***

What it does: A one-prompt keyword that asks Claude for the deepest reasoning available, regardless of session-level `/effort` setting.

When to use it: For prompts where you'd otherwise wait an hour to ask a colleague. Costs 5-10x a normal prompt.

Example: "`ultrathink`, *explain why the LinkedIn refresher is intermittent and propose three architecturally different fixes.*" Deeply reasoned response.

Related: `/effort` (session-level), `/model` (heavier model for the same prompt is sometimes the cheaper alternative).

Taught in: *Chapter 25. (Keyword, not a slash command: type it inline in any prompt.)*

When a command behaves unexpectedly

A reference appendix has no failure beat, but if something here doesn't match what your terminal does:

> **Note**
>
> *Check the version stamp at the top of the appendix. The Claude Code surface evolves; commands may have been renamed or restructured. Check the errata page (*`github.com/codecoincognition/claude-code-4-non-programmers-v2/blob/main/ERRATA.md`*) for current state. If the entry here doesn't match, the errata page wins.*

If a command is missing entirely from this appendix and you've seen it referenced in the chapters, the errata page also tracks additions. The book ships with what was canonical at print; the errata page tracks deltas.

How the families compose

The families table is the reading aid for finding the right neighborhood before drilling into a specific entry.

Family	Members	What they share
Navigation	`/rewind`, `/branch`, `/fork`, `--resume`	Move through session history
Mid-task control	`/btw`, `Ctrl + G`	Adjust in-flight work
Awareness	`/cost`, `/context`, `/compact`	Sense the session
Intensity	`/model`, `/effort`, `ultrathink`	Tune reasoning depth and cost
Modes	`/plan`, `/auto` *	Set behavioral posture
Review	`/simplify`, `/code-review`, `/security-review`, `/sandbox`, `/batch`, `/review-mine` *	Audit existing artifacts
Self-improvement	`/insights`	Observe and propose
Persistence	`/schedule`, `/loop`, `/goal`	Keep work going past a single turn: clock-shaped, session-shaped, or condition-shaped
Discovery	`/plugin`, `/skills`	Find and inspect capabilities

Common compositions: - `/insights` → (manual distillation prompt) → `/auto` (observe, file as skill, let auto pick it next time) - `/security-review` → `/sandbox` (audit static, then verify behavior) - `/cost` → `/context` → `/compact` (situational awareness, then act on it) - `/branch` → work in copy → `claude --resume <parent>` (try an alternative without committing the main thread)

The table is a reading aid, not a taxonomy: some commands belong to more than one family. `/auto` is a Mode that picks among Review commands. `/schedule` is Scheduling but it also drives every long-running Self-improvement loop. Use the family to find the right neighborhood, not as a rigid index. Entries marked * are personalized commands authored in this book (*Chapter 26*), not Claude Code built-ins.